Portfolio Website — Product Requirements Document (PRD)

Owner: Ayush Raj **Prepared for:** build in Claude Code **Version:** 1.0 — 2026-08-23 **Design direction:** “Wire Format” **Build:** Static single-page site, deployed on GitHub Pages

---

1. Purpose & goal

Build a personal portfolio website that makes a strong first impression on a technical visitor and convinces them, within about ten seconds, that Ayush genuinely does security work at two layers most students skip: the packet and the exploit. The site then lets the visitor drill down into real architecture and decision reasoning for each project.

Primary audience: internship recruiters and security engineers. Secondary: hackathon judges.

The one sentence the visitor should leave with: *“This person works one layer deeper than a demo — he reads packets and he breaks real applications, and he can defend every decision.”*

Success criteria - Loads fast (static, no heavy framework), scores well on Lighthouse. - The hero communicates the thesis before any scrolling. - Every project can be expanded from a one-line summary into a full technical dissection. - The credential hierarchy reads correctly to a technical reviewer (CND is clearly the anchor; workshops/participation are visibly secondary). - Fully responsive, keyboard accessible, respects reduced motion.

---

2. Design system — “Wire Format”

The concept: the page is styled like a printed protocol specification (RFC / packet header). Cool “engineering paper” palette, technical-documentation typography, and a hero rendered as an RFC 791 IP-header bit-field diagram whose fields hold Ayush’s identity. Deliberately avoids the neon-green-on-black hacker cliché (which is also a generic AI-design default).

2.1 Colour tokens

```
--ground:      #E9ECF1   /* page background — cool engineering paper */
--panel:       #FFFFFF   /* cards / raised surfaces */
--panel-alt:   #F4F6F9   /* subtle striped rows, code blocks */
--ink:         #101319   /* primary text, near-black */
--ink-soft:    #47505E   /* secondary text */
--rule:        #A7B0BD   /* hairlines, bit-field borders */
--rule-soft:   #CDD4DE   /* light grid lines */
--signal:      #5B3DF5   /* electric violet — INTERACTIVE / live elements ONLY */
--signal-weak: #ECE9FE   /* violet tint for hover backgrounds */
```

/* Severity scale — used ONLY for the VAPT findings, encodes real CVSS bands */
--crit: #C81E3A /* critical */
--high: #E06C00 /* high */
--med: #B58900 /* medium */
--low: #6B7280 /* low / informational */

Rule: violet (--signal) is reserved for things the user can interact with or that are "live" (links, the expand control, hover states). Do not use it decoratively. Severity colours appear only inside the VAPT project.

2.2 Typography

Load from Google Fonts (all free, GitHub-Pages friendly): - **Display:** Archivo (weights 600/700/800) — headlines, the name, section titles. Tight letter-spacing. - **Body:** IBM Plex Sans (400/500/600) — prose. - **Data / mono:** IBM Plex Mono (400/500) — every piece of "data": field values, labels, eyebrows, CVSS numbers, tags, code, timestamps, the bit-field.

Type scale:

Role	Font	Size	Weight	Notes
Hero display	Archivo	clamp(2.5rem, 6vw, 5rem)	800	letter-spacing -0.02em
H2 section	Archivo	clamp(1.6rem, 3vw, 2.1rem)	700	
H3	Archivo	1.25rem	600	
Body	IBM Plex Sans	1rem	400	line-height 1.65
Data / label	IBM Plex Mono	0.8125rem	500	uppercase labels, letter-spacing 0.02em
Eyebrow / micro	IBM Plex Mono	0.6875rem	500	UPPERCASE, letter-spacing 0.15em

2.3 Layout & spacing

- Max content width ~1080px, centered, generous side margins (min 20px on mobile).
- 8px spacing base (8/16/24/40/64/96).
- Section rhythm: ~96px vertical padding desktop, ~56px mobile.
- Cards: 1px --rule border, 0 or 2px border-radius (keep it "spec sheet" crisp, not rounded/soft), white panel background.
- Hairline dividers between sections using --rule-soft.

2.4 Structural metaphor (encodes meaning, not decoration)

Treat the whole page like a packet: the **header** (hero bit-field) carries identity fields, then the **payload** is the content. Each section gets a short mono "field label" in its header, drawn

from packet vocabulary, used meaningfully rather than as decorative 01/02/03 numbering:

- SRC → About
- PAYLOAD → Projects
- OPTIONS → Credentials & skills
- TTL → Experience
- FLAGS → Contact ("available for hire")

2.5 Motion

- **Page load (one orchestrated moment):** the bit-field fields populate left-to-right quickly, like a header being written. ~600ms total.
- **Hover micro-interactions:** bit-field fields highlight and show a tooltip; project rows lift 1px; links get a violet underline.
- **Project expand/collapse:** smooth height + fade, ~250ms.
- **Respect prefers-reduced-motion: reduce** — disable the load animation and expand transitions; render everything in final state instantly.

---

3. The signature element — RFC 791 bit-field hero

The one thing the site is remembered by. A monospace grid with a 0…31 bit ruler across the top and boxed fields below, exactly like the IP header diagram in RFC 791 — but the fields hold Ayush's identity.

Desktop reference (ASCII; render as styled HTML boxes, not literal text art):

```
 0               1               2               3
 0 1 2 3 4 5 6 7 8 9 0 1 2 3 4 5 6 7 8 9 0 1 2 3 4 5 6 7 8 9 0 1
+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+
|  Ver  |  Role |          Name:  AYUSH  RAJ                    |
+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+
|   Focus: Network Defense  &  Offensive Web    |    Flags      |
+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+
|            Source:  Sharda University  (B.Tech CS)           |
+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+
|        Options:  EC-Council CND  |  Scapy  |  Burp Suite     |
+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+
```

Field meanings (tooltip on hover/focus, written like an RFC): - Ver → "Version 2 — 2nd-year CS." - Role → "Security analyst in the making." - Name → "Ayush Raj." - Focus → "Network defense and offensive web-app security." - Flags → "Set: available for hire." (this flag animates/pulses subtly in --signal) - Source → "B.Tech Computer Science, Sharda University." - Options → "Certified Network Defender; packet analysis with Scapy; web testing with Burp Suite."

Supporting copy directly below the diagram: - Eyebrow (mono): AYUSH RAJ // SECURITY - Headline (Archivo): **I work one layer deeper than the demo.** - Sub (Plex

Sans): “Second-year CS student and security practitioner. I build packet-level detection tools, and I break real web apps to understand how they fail. EC-Council Certified Network Defender.” - Primary CTA: **Read the dissections** ↓ (smooth-scroll to Projects) - Secondary: GitHub · LinkedIn

Mobile fallback (critical): the 64-column bit ruler cannot fit narrow screens. Below ~640px, drop the ruler and render each field as a stacked full-width “field card” — a mono label on top (NAME, FOCUS, FLAGS…) and the value below, keeping the boxed/spec aesthetic without horizontal scrolling. Never allow horizontal scroll.

---

4. Information architecture

Single page, anchored sections, sticky slim top nav (mono links: SRC · PAYLOAD · OPTIONS · TTL · FLAGS).

1. Hero (bit-field)
2. About (SRC)
3. Projects (PAYLOAD) — the core, with drill-downs
4. Credentials & Skills (OPTIONS)
5. Experience (TTL)
6. Contact (FLAGS) + footer

---

5. Section content & copy

5.1 About (SRC)

> I’m a second-year Computer Science student at Sharda University focused on network defense and offensive web security. My work sits at two layers most student projects skip: the packet and the exploit. On the defensive side I build tools that read raw traffic — live capture, statistical anomaly detection, host-based behavioral firewalls. On the offensive side I test real applications the way an attacker would, then document what I find to professional standard. I care less about shipping a demo than about being able to defend every decision inside it.

5.2 Projects (PAYLOAD) — drill-down component

Each project renders as a collapsed **row/card** showing: title, one-line summary, stack tags (mono chips), and a status pill (COMPLETED / IN PROGRESS / LIVE). A visible expand control (▸) opens a **dissection tree** with these labelled parts (only include the parts that apply to each project):

▸ Problem · ▸ Approach / Architecture · ▸ Pipeline · ▸ Tradeoffs & decisions · ▸ Failure modes / limits · ▸ Metrics · ▸ Links

Interaction: expandable via click and keyboard (Enter/Space), aria-expanded on the control, aria-controls pointing to the panel. Only concern: keep CSS selector specificity clean so section paddings don't cancel (see §9).

Order (strongest / most recruiter-relevant first):

PROJECT 1 — Online Assessment Platform: Security Assessment (VAPT)

Status: COMPLETED · Tags: Burp Suite · Manual VAPT · OWASP · CVSS · Web

Metrics strip (render as coloured pills using the severity scale): 1 CRITICAL (CVSS 9.8) · 1 HIGH · 3 MEDIUM · 4 LOW · 2 INFO

▸ **Problem** Online proctored coding platforms are a uniquely dangerous attack surface: by design they accept untrusted code, compile it, and run it, while also trying to enforce exam integrity. I performed a full vulnerability assessment and penetration test of one such platform (a Next.js-based online assessment system), end to end.

▸ **Approach** Manual testing driven through Burp Suite (Proxy, Repeater, Intruder) alongside authenticated scanning. Every finding was mapped to the OWASP Top 10 (2021) and a CWE, scored with CVSS, and documented with a reproducible proof-of-concept and a concrete remediation.

▸ **Key findings (redacted to technique)** - **Critical — Remote Code Execution.** The backend compiled and executed submitted C with no sandbox. A payload invoking an OS command executed at the operating-system level; impact was proven by a deliberate timed delay in the server's response. Remediation: isolated execution (containers with dropped capabilities, seccomp/AppArmor), restricted compiler surface. - **High — Vertical privilege escalation.** Role authorization was enforced only in client-side routing, so a low-privilege session token replayed against admin/teacher endpoints returned protected data. Remediation: server-side RBAC on every protected route. - **Medium —** CSRF on the submission endpoint; a proctoring anti-cheat bypass (posting code straight to the API, past the browser's paste/copy listeners); no rate limiting or lockout on login. - **Low / Info —** missing anti-clickjacking headers, reflected user input, HSTS not enforced, cacheable sensitive responses.

▸ **What it demonstrates** I can test a real, complex application methodically, prove impact instead of asserting it, and communicate risk in the language both a security team and a developer need.

▸ **Responsible disclosure (show this on the site — it is itself a credential)** > Target identity, live host, credentials, and working exploit payloads are deliberately withheld. Findings were reported to the system's owner. This page describes methodology and technique only.

No public repo/report link by design.

PROJECT 2 — OmniSniff: Network Traffic Analyzer & Intrusion Detector

Status: COMPLETED · Tags: Python · Scapy · Flask-SocketIO · Detection

▸ **Problem** Most student security projects live at the web layer. Real attacks often show up on the wire first — a port scan, a sudden traffic spike, a host talking to something it never talks to. Reading that requires working at packet level, which off-the-shelf dashboards abstract away.

▸ **Approach / Architecture** Live capture with Scapy feeds a detection layer that uses rolling-average baselines and sliding-window analysis to separate normal variance from a scan or spike. Alerts stream to a Flask-SocketIO dashboard in real time and export to CSV/HTML reports.

▸ **Pipeline** Capture (Scapy sniff) → per-flow feature extraction → sliding-window aggregation → rolling-average baseline comparison → alerting → live dashboard + report export.

▸ **Tradeoffs & decisions** Chose rolling-average / sliding-window statistics over a heavyweight ML model: real-time, explainable, and needs no training data. The cost is that it catches statistical anomalies, not semantically novel attacks — a deliberate, documented trade.

▸ **Failure modes / limits** Capture throughput is bounded by Scapy/Python, so it's designed for monitoring representative traffic, not saturating a gigabit link. That ceiling is understood, and I can explain the architecture (AF_PACKET, kernel-bypass, eBPF/XDP) that would lift it.

▸ **Links** — GitHub repo *(URL needed — see §11)*

PROJECT 3 — Application-Context Aware Firewall

Status: IN PROGRESS · Tags: Python · Threat Intel · Behavioral Detection · Host Security

▸ **Problem** Traditional firewalls decide on ports and IPs. They can't tell that a trusted application has been hijacked: if Chrome is allowed out and malware rides Chrome, the traffic passes. That gap is how a lot of real exfiltration and malware command-and-control succeeds.

▸ **Approach — three innovations** - **Live threat intelligence.** Every outbound destination is checked in real time against AbuseIPDB / VirusTotal; known-malicious IPs are blocked instantly. - **Behavioral anomaly detection.** The firewall learns each application's normal traffic profile and flags exfiltration-shaped deviations — e.g. a document editor suddenly pushing hundreds of MB to an unknown host. - **Process-tree lineage.** It inspects how a process was spawned to catch masquerading: a chrome.exe launched silently by an Office macro is treated very differently from one the user opened.

▸ **Open questions (ongoing)** Tuning false positives from behavioral learning; free-tier threat-intel rate limits; where enforcement should live (userland vs kernel hooks).

▸ **What it demonstrates** I think about detection at the layer attackers actually operate — process and behavior — not just the perimeter.

▸ **Links** — GitHub repo *(URL needed — see §11)*

PROJECT 4 — Evently: Full-Stack Event Booking & Management Platform

Status: LIVE · Tags: React · Node/Express · MongoDB · JWT · RBAC Credit: built with a collaborator (@ayushjaiswal36937-dev).

▸ **Problem** I wanted to build a real multi-role web application end to end — the kind of system I test from the offensive side — so I'd understand first-hand how authentication, authorization, and state get implemented, and where they break.

▸ **Approach** MERN stack. JWT authentication with OTP verification; role-based access control across three roles (attendee / organizer / admin) enforced server-side; event CRUD with image upload (Multer); booking with unique references and QR ticket stubs; HTML confirmation email; wishlist and reviews; analytics dashboards with CSV export. ~20 REST endpoints across User, Event, and Booking models.

▸ **Security-conscious build (the through-line)** > Having found broken access control in a penetration test — where role checks lived only in the client — I built Evently's authorization as server-side middleware on every protected route.

▸ **Tradeoffs & decisions** MongoDB for schema flexibility and iteration speed; JWT for stateless auth; auto-publish on event creation for a frictionless organizer flow.

▸ **Links** — Live: event-booking-system-dun.vercel.app · GitHub: github.com/Aayushraj2121/Event_booking

---

PROJECT 5 — Drift-Sense: Synthetic Data Generation + Siamese CNN

Status: COMPLETED · Tags: Python · OpenCV · Siamese CNN · Synthetic Data

▸ **Problem** The hardest part of many ML problems isn't the model — it's that no labelled dataset exists. For wafer-defect detection there was none, so I generated one.

▸ **Approach** Built a synthetic SEM wafer-image generator to produce labelled defect / non-defect pairs, trained a Siamese CNN using normalized cross-correlation for similarity, and validated against a benchmark harness of 30+ test cases.

▸ **Why it matters for security** The same instinct — manufacture representative data and a rigorous evaluation harness when real data doesn't exist — is exactly what unsupervised threat detection needs. It's the bridge between my ML work and my security work.

▸ **Links** — GitHub repo *(URL needed — see §11)*

5.3 Credentials & Skills (OPTIONS)

Hierarchy is deliberate — do NOT render these as four equal cards.

Anchor credential (prominent card, with badge art + verify link): - **EC-Council Certified Network Defender (CND)** — ANSI-accredited. Credential no. ECC3120469785. Issued 21 May 2026, valid three years. Badge image supplied (CND_9AF22971F673.png). **Verify** → *(Credly/Aspen URL needed — §11).*

Secondary strip (smaller, single row): - Advanced Certificate Program in Cyber Security & Digital Forensics (CSDF-2026) — Sharda University, Center for Cyber Security & Cryptology, Feb 2026.

Hands-on practice (framed as practice, not a certificate): - **PortSwigger Web Security Academy — 47 labs solved:** 25 Apprentice, 20 Practitioner, 2 Expert. (Lead with "20 Practitioner-level labs"; do not surface the gamified "Newbie" rank label.) *Profile URL optional — §11.*

Activity line: - Smart India Hackathon 2025 — internal round, team "Loop Coder", Sept 2025.

Optional: original certificate scans behind a click-to-view lightbox.

Skills — grouped by what was built with each (no progress bars):

Group	Tools / concepts	Evidence
Offensive / Web App Security	Burp Suite, manual VAPT, OWASP Top 10, CWE, CVSS, Nmap, Metasploit, Hydra, Kali Linux	VAPT assessment (9 findings); 47 PortSwigger labs
Network / Packet Analysis	Python, Scapy, Wireshark, tcpdump, Flask-SocketIO	OmniSniff
Detection & Host Security	Behavioral anomaly detection, process-tree lineage, threat-intel APIs (AbuseIPDB, VirusTotal)	Application-Context Aware Firewall
Full-Stack Engineering	React, Node.js, Express, MongoDB, JWT, RBAC, REST	Evently
ML / Data	Synthetic data generation, Siamese CNN, OpenCV, evaluation harnesses	Drift-Sense
Languages	Python (primary), JavaScript, C++, C, Java	across projects

5.4 Experience (TTL)

- **Cybersecurity Intern — SkillOrbit / Sofzenix IT Solutions** (May – Jun 2026). Worked on defensive security best practices and vulnerability remediation documentation. *(Confirm exact start month — §11.)*

5.5 Contact (FLAGS) + footer

- Headline: **Flags: available for hire.**
- Line: "Open to security internships. The fastest way to reach me is email."
- Links: ayushraj36937@gmail.com · GitHub github.com/Aayushraj2121 · LinkedIn linkedin.com/in/ayush-raj-2294a0385
- Footer: mono line, e.g. EOF // built by Ayush Raj // 2026.

---

6. Tech stack & build

- **Static single-page site.** Semantic HTML5 + vanilla CSS (custom properties) + vanilla JS. No framework, no build step — keeps the repo clean/readable (itself a signal to technical reviewers) and deploys to GitHub Pages with zero config.
- Fonts via Google Fonts <link> (Archivo, IBM Plex Sans, IBM Plex Mono).
- Suggested files: index.html, styles.css, main.js, /assets/ (badge, favicon, OG image, optional cert scans).
- *(If you prefer components, Astro would also deploy statically to Pages — but plain static is recommended here.)*

---

7. Accessibility & quality floor (non-negotiable)

- Responsive down to 320px; no horizontal scroll anywhere (watch the bit-field — use the mobile fallback in §3).
- Visible keyboard focus on every interactive element; project drill-downs operable by keyboard with correct aria-expanded/aria-controls.
- Colour contrast: body text --ink on --ground and on --panel must meet WCAG AA. Verify severity-pill text contrast.
- prefers-reduced-motion: reduce disables load animation and expand transitions.
- Semantic landmarks (header, nav, main, section, footer), alt text on the badge image, sensible heading order.

---

8. Deployment — GitHub Pages

1. Create a repo named **Aayushraj2121.github.io** (a user-site repo publishes at the root URL https://aayushraj2121.github.io).
2. Put index.html at the repo root.
3. Push to main.
4. Repo → Settings → Pages → Source: "Deploy from a branch" → main / root.
5. Wait ~1 min; the site is live at https://aayushraj2121.github.io.

6. *(Optional later: custom domain via a CNAME file.)*

9. Build notes for Claude Code

- **CSS specificity trap:** avoid element-plus-class selectors that cancel each other on section spacing (e.g. .section vs a section rule). Use a single consistent class strategy for section padding so vertical rhythm doesn't collapse.
- Spend visual boldness in ONE place — the bit-field hero. Keep everything else quiet and disciplined.
- Build the severity pills and status pills as small reusable components.
- Test the expand/collapse and the mobile bit-field fallback before anything else.

10. Pre-launch checklist

- ☐ Confirm Evently's RBAC is real server-side middleware on every protected route (turns the "I build auth right" story from claim into fact).
- ☐ Ensure **no real .env / secrets** are committed in the Evently repo (README ships sample creds + JWT_SECRET — placeholders are fine, live secrets are not).
- ☐ Tidy READMEs on every featured repo — the site will drive recruiter traffic to them.
- ☐ Re-confirm the VAPT section contains no live host, no credentials, no working payloads.
- ☐ Lighthouse pass (performance + accessibility).

11. Open items needed from Ayush

1. **Repo URLs** for OmniSniff, Application-Context Aware Firewall, and Drift-Sense (Evently already have; VAPT has no public link by design).
2. **CND verification URL** (Credly or EC-Council Aspen) for the badge click-through.
3. **Headshot decision** — the supplied photo looks AI-generated; for an authenticity-first security site, a real photo or a clean monogram/avatar is safer. Choose one.
4. **PortSwigger profile URL** (optional — only if you want it linked).
5. **Internship start month** — certificate says 1 May 2026; confirm.
6. Any project screenshots or architecture diagrams you want embedded in the drill-downs.

Appendix A — Alternative design directions (not chosen)

- **Capture Session** — dark Wireshark three-pane instrument chrome; projects as packet rows expanding into a dissection tree.
- **Case File** — cold archival-paper forensic look; evidence records, hashes, chain-of-custody, serif body.

Selected: Wire Format.